Practice Midterm Worked Solutions

English answer key with calculations, lane-level index traces, code fixes, and exam-oriented reasoning.

HOW TO USE THIS KEY

Try each question first. Then compare the final answer, the reasoning, and the exam takeaway. For code questions, write the indices for lanes 0-3 before deciding whether an access pattern is coalesced.

QUICK ANSWER KEY

Q	Answer
1	(a) T, (b) T, (c) T, (d) T, (e) T, (f) F, (g) F, (h) F, (i) T, (j) F, (k) T, (l) T, (m) F, (n) T
2	(a) Multiple hardware threads share one core. (b) Every rank receives all contributed chunks. (c) Read-only GPU cache optimized for spatial access.
3	(a) 4.71x on 8 processors. (b) Maximum 10x.
4	i = 2 x blockIdx.x x blockDim.x + threadIdx.x; second index j = i + blockDim.x.
5	Expected course answer: BLOCK_SIZE = 1,...,5. Add __syncthreads() after the shared-memory store for all sizes.
6	(a). It gives unit-stride first accesses across neighboring lanes.
7	No tiling: N requests per input element. T x T tiling: N/T requests per input element.
8	1000 logical shared-memory instances over the kernel lifetime - one per block.
9	Shared memory permits cross-thread sharing and reuse; registers are private to one thread.

Important source typos: Q5 mixes BLOCK_WIDTH and BLOCK_SIZE; the solution treats them as the same tile dimension. Q6 writes B[I]; the intended expression is B[i].

QUESTION 1 - TRUE / FALSE WITH REASONS

Item	Answer	Reason
(a)	**True**	GPU scratchpad/shared memory is explicitly accessed by load and store instructions.
(b)	**True**	GPU cores spend less hardware on complex single-thread control and more on throughput.
(c)	**True**	A barrier is collective: every required participant must arrive before any may continue.
(d)	**True**	Gustafson models scaled speedup as the problem size grows with the processor count.
(e)	**True**	Roofline compares the compute roof P_{peak} with the bandwidth roof B x I.
(f)	**False**	Increasing I helps only until the compute roof is reached; performance then saturates.
(g)	**False**	Copying, synchronization, bank conflicts, or too little reuse can erase the benefit.
(h)	**False**	Hardware handles divergence functionally, but different paths are typically serialized with lane masks.
(i)	**True**	cudaMallocManaged() allocates Unified Memory accessible from CPU and GPU.
(j)	**False**	If all active lanes in a warp take the same path, that branch does not diverge.
(k)	**True**	Under the course model, superscalar out-of-order hardware dynamically selects ready independent instructions to issue.
(l)	**True**	One vector instruction controls multiple lanes, reducing instruction fetch/decode demand per data element.
(m)	**False**	Warp/wavefront width is fixed by the hardware architecture, not chosen by the program.
(n)	**True**	The course material describes Arm Valhall as supporting hardware dynamic instruction scheduling.

Exam takeaway: statements containing always, without penalty, or controlled by software deserve an immediate counterexample check.

QUESTIONS 2 AND 3 - DEFINITIONS AND AMDAHL

Question 2 - concise definitions

2(a) Simultaneous multithreading (SMT)

A single physical CPU core keeps architectural state for multiple hardware threads and can issue instructions from different threads in the same cycle. This improves utilization when one thread is stalled or lacks enough instruction-level parallelism.

2(b) All-gather communication

Every process contributes a local chunk, and every process receives the concatenation of all chunks. Unlike Gather, the complete result is delivered to every rank; unlike Allreduce, the values are collected rather than arithmetically combined.

2(c) GPU texture cache

A read-only GPU cache/path optimized for spatial locality, especially 2D access patterns. Texture hardware may also provide addressing and filtering support; for this exam, remember read-only spatial caching.

Question 3 - Amdahl's law

Serial fraction s=0.10, parallel fraction 1-s=0.90, and p=8 processors.

$$S(p) = 1 / [s + (1-s)/p]$$

$$S(8) = 1 / [0.10 + 0.90/8] = 1 / 0.2125 = 4.70588$$

3(a) Final answer

The expected speedup is approximately 4.71x.

As p approaches infinity, the parallel term (1-s)/p approaches zero:

$$S(infinity) = 1/s = 1/0.10 = 10$$

3(b) Final answer

The maximum possible speedup is 10x.

Sanity check: the answer on 8 processors must be below both 8x and the 10x serial-fraction ceiling.

QUESTIONS 4 AND 6 - INDEX MAPPING AND COALESCING

Question 4 - two contiguous sections per block

One block covers 2 x blockDim.x consecutive elements. Therefore the block base advances by 2 x blockDim.x, while neighboring threads differ by one:

$$i = 2 \times blockIdx.x \times blockDim.x + threadIdx.x$$

The second section begins one block width later:

$$j = i + blockDim.x$$

```
int i = 2 * blockIdx.x * blockDim.x + threadIdx.x;
int j = i + blockDim.x;
if (i < n) C[i] = A[i] + B[i];
if (j < n) C[j] = A[j] + B[j];
```

Question 6 - multiple choice

Final answer: (a)

int i = (blockIdx.x * blockDim.x) * 2 + threadIdx.x;

Let blockDim.x=128 and blockIdx.x=1. Compare the first memory instruction across lanes 0-3:

Choice	Indices for lanes 0-3	Pattern	Result
(a)	256, 257, 258, 259	Unit stride	Coalescing-friendly
(b)	256, 258, 260, 262	Stride 2	More memory segments

Coalescing is judged across lanes for one memory instruction. Choice (b) gives each thread adjacent personal work over time, but its first accesses are not adjacent across the warp.

Fast method: substitute threadIdx.x = 0,1,2,3. The correct option should produce consecutive first indices.

QUESTION 5 - TILE TRANSPOSE AND SYNCHRONIZATION

The shared-memory write is performed by thread (tx,ty), but the next statement reads the transposed location written by thread (ty,tx). This is a producer-consumer dependency between threads.

$$\text{Number of threads in the block} = \text{BLOCK_SIZE}^2$$

Under the course's warp-synchronous model, the unsynchronized code works only when the entire block fits in one NVIDIA warp:

$$\text{BLOCK_SIZE}^2 <= 32 \Rightarrow \text{BLOCK_SIZE} <= \text{floor}(\text{sqrt}(32)) = 5$$

5(a) Expected course answer

BLOCK_SIZE = 1, 2, 3, 4, or 5. For BLOCK_SIZE >= 6, the block has more than 32 threads, so values may be produced and consumed by different warps without an ordering guarantee.

Insert a block-wide barrier after every thread stores its shared-memory element and before any thread reads the transposed element:

```
blockA[threadIdx.y][threadIdx.x] = A_elements[baseIdx];
__syncthreads();
A_elements[baseIdx] = blockA[threadIdx.x][threadIdx.y];
```

5(b) Fix

Add __syncthreads() between the shared-memory store and the transposed read. Then every thread in the block completes the producer phase before any thread begins the consumer phase.

Portable CUDA note

Modern CUDA permits more independent scheduling within a warp than older warp-synchronous reasoning assumed. Production code should use explicit synchronization even when BLOCK_SIZE <= 5. The block-wide barrier above is safe for every allowed size and is the answer to write for part (b).

Barrier rule: all threads in the block must reach __syncthreads() consistently. A block barrier does not synchronize different blocks.

QUESTIONS 7-9 - MEMORY REQUESTS AND SHARING

Question 7 - matrix multiplication request count

For $C_{ij} = \text{sum}_{k=0}^{N-1} A_{ik}B_{kj}$, one element A_{ik} is needed by all N output columns j. Similarly, one B_{kj} is needed by all N output rows i.

7(a) No tiling

Each input element is requested N times. Across both input matrices, the naive kernel issues $2N^3$ global-memory requests.

With a T x T tile, one loaded element is reused by T threads in the block. The same element must be loaded once for each group of T outputs along the other dimension:

requests per input element = N / T

7(b) T x T tiling

Each input element is requested N/T times, assuming T divides N. Total requests become $2N^3/T$, a factor-T reduction.

The question counts requests to the global-memory system including caches, so use the algorithmic load count rather than trying to predict DRAM cache hits.

Question 8 - shared-memory instances

Final answer: 1000

Shared memory is allocated per thread block, not per thread and not once per grid. Therefore 1000 blocks create 1000 logical versions over the kernel's lifetime. Only resident blocks exist simultaneously; the physical storage is reused as blocks finish.

Question 9 - why shared memory instead of registers?

Core reason: cross-thread sharing

Registers are private to one thread. Shared memory is visible to every thread in the block, so one thread can load a value from global memory and other threads can reuse it. This reduces redundant global loads and enables cooperative algorithms such as tiled matrix multiplication or transpose.

- Shared memory also supports indexing patterns that are awkward or impossible with per-thread registers.
- The tradeoff is explicit synchronization and possible bank conflicts.

If the question says register/shared capacity is not a problem, do not answer only 'shared memory is larger.' The requested distinction is visibility and reuse across threads.

FINAL EXAM ROUTINE

For formulas: identify the model, substitute symbolically, then check limits. For CUDA indexing: list lane 0-3 addresses. For synchronization: identify the producer, consumer, and scope. For memory questions: distinguish per-thread registers, per-block shared memory, and grid-wide global memory.